

Flutter + C++ Cross-Platform Application Engineering Skill

0. Skill 定位

本 Skill 用于指导 AI Coding Agent 开发基于：

- Flutter
- C++
- Dart
- FFI / Platform Channel
- iOS
- Android
- Linux

的跨平台应用。

目标是构建：

Flutter 跨平台 UI + C++ 跨平台核心能力 + 原生平台 API / UI 适配

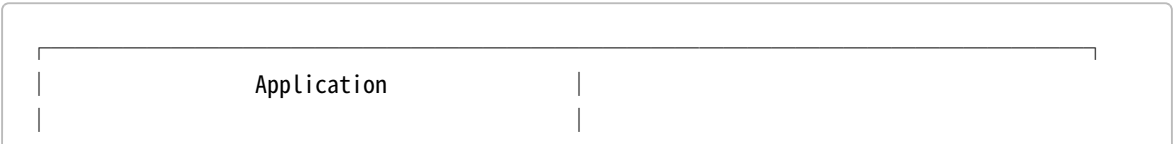
的现代跨平台应用架构。

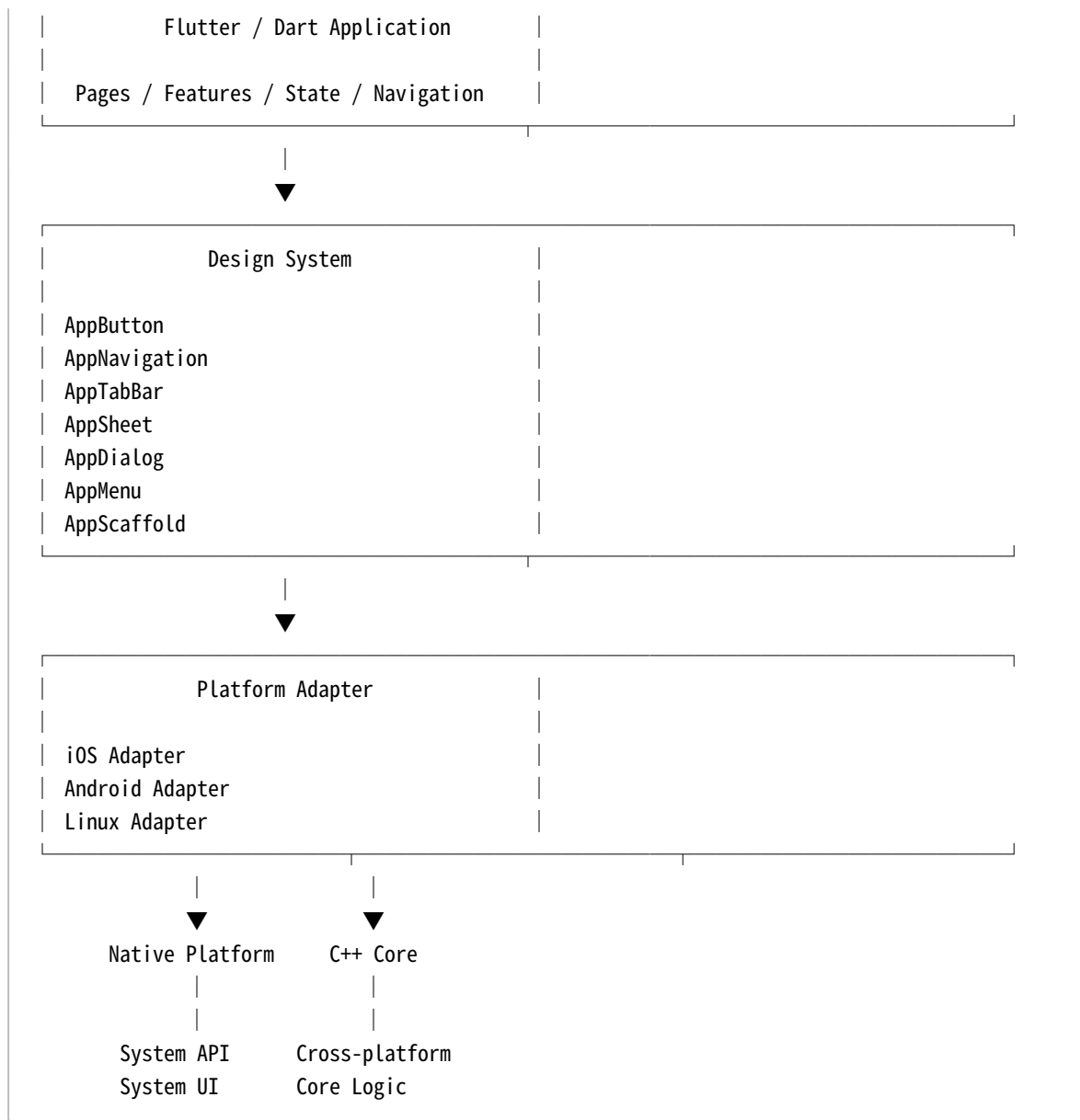
本规范强调：

1. 优先遵循各平台官方设计规范。
2. 优先使用系统提供的 UI 能力。
3. Flutter 负责跨平台 UI 结构和业务交互。
4. C++ 负责跨平台核心逻辑和性能敏感功能。
5. 原生代码负责平台特有功能。
6. 不为了代码统一而牺牲平台原生体验。
7. 不允许在业务代码中散落平台判断。
8. 不允许直接在业务页面调用平台原生 API。
9. 所有平台差异必须集中到 Platform Adapter。
10. 所有通用 UI 必须通过 Design System 抽象。
11. AI 必须优先查阅官方文档后再实现平台特性。
12. 不得凭经验伪造或猜测系统 API。

1. 总体架构

采用四层架构：





2. 三个平台的职责

2.1 Flutter

Flutter 负责：

- 页面布局
- 页面路由
- 页面状态
- UI 交互
- Design System
- 跨平台页面结构
- 业务流程编排
- 用户输入

- 数据展示
- 动画
- 响应式布局
- 跨平台 UI 逻辑

Flutter 不负责：

- 复杂底层算法
- 高性能图像处理
- 大规模文件扫描
- 大规模数据库计算
- 平台专属系统 API
- 平台专属系统 UI
- 直接管理底层硬件

2.2 C++

C++ 负责：

- 跨平台核心逻辑
- 高性能算法
- 图像处理
- 视频处理
- 文件扫描
- 文件索引
- 数据库核心访问
- 搜索索引
- 大量数据处理
- CPU 密集型任务
- 内存敏感任务
- 可复用跨平台核心库

C++ 代码必须尽量做到：

```
Core C++
|
├── iOS
├── Android
└── Linux
```

共享同一套核心实现。

禁止在 C++ Core 中直接依赖：

```
UIKit
SwiftUI
Android SDK
```

平台代码必须通过 Adapter 隔离。

2.3 Native Platform

原生平台负责：

iOS

- Apple Photos API
- 文件系统 API
- 系统权限
- 系统分享
- 系统文档选择器
- 系统通知
- 系统生命周期
- 系统 UI
- Apple 官方设计规范
- Liquid Glass 等平台视觉能力

Android

- MediaStore
- Storage Access Framework
- Android 权限
- Android Share
- Android 生命周期
- Android 系统通知
- Material Design
- 系统 UI

Linux

- Linux 文件系统
 - GTK / 系统能力（必要时）
 - Desktop integration
 - 系统文件选择器
 - DBus
 - Desktop notification
-

3. 核心原则

原则 1：Platform First

任何涉及平台 UI 的功能：

先检查平台官方能力
↓
再检查 Flutter 官方能力
↓
最后才自行实现

优先级：

Official Native API
↓
Official Flutter API
↓
Flutter Package
↓
自定义实现

禁止：

已经存在官方系统能力，却重新模拟一个系统组件。

原则 2：Semantic UI

业务代码描述 UI 的语义，而不是平台实现。

业务代码：

```
AppButton(  
  label: '删除',  
  variant: AppButtonVariant.destructive,  
  onPressed: delete,  
);
```

禁止：

```
if (Platform.isIOS) {  
  CupertinoButton(...);  
} else {  
  ElevatedButton(...);  
}
```

平台适配由 Design System 完成。

原则 3：Native Experience

不同平台不要求 UI 完全一致。

允许：

```
iOS
→ Apple Navigation
→ Liquid Glass
→ Sheet
→ Context Menu

Android
→ Material Navigation
→ Material Bottom Sheet
→ Material Dialog

Linux
→ Desktop Sidebar
→ Desktop Menu
→ Desktop Dialog
```

目标不是：

所有平台看起来一样。

目标是：

所有平台具有一致的产品逻辑，同时遵循各自平台的官方设计语言。

4. Design System

所有业务页面必须优先使用 Design System。

推荐：

```
design_system/
├── buttons/
│   ├── app_button.dart
│   └── app_icon_button.dart
├── navigation/
│   ├── app_navigation.dart
│   ├── app_navigation_bar.dart
│   ├── app_navigation_rail.dart
│   └── app_tab_bar.dart
```

```
|
|
|—— surfaces/
|   |—— app_scaffold.dart
|   |—— app_sheet.dart
|   |—— app_dialog.dart
|
|—— menus/
|   |—— app_menu.dart
|
|—— inputs/
|   |—— app_text_field.dart
|   |—— app_switch.dart
|   |—— app_checkbox.dart
|   |—— app_slider.dart
|
|—— platform/
|   |—— ios/
|   |—— android/
|   |—— linux/
```

5. AppButton

业务代码：

```
AppButton(
  label: '删除',
  variant: AppButtonVariant.destructive,
  onPressed: delete,
);
```

平台：

```
iOS
→ Apple Button
→ Liquid Glass compatible

Android
→ Material Button

Linux
→ Desktop Button
```

业务层不得知道实际实现。

6. AppNavigation

AppNavigation 表示 App 一级导航。

手机：

Bottom Navigation

平板：

Sidebar

桌面：

NavigationRail / Sidebar

API 保持统一：

```
AppNavigation(  
  items: [...],  
  currentIndex: currentIndex,  
  onChanged: onChanged,  
);
```

不要固定命名为：

AppBottomNavigationBar

因为桌面端可能不是 Bottom Navigation。

7. AppTabBar

AppTabBar 只表示当前页面内部的 Tab。

例如：

全部
人物
地点
收藏

不能用 AppTabBar 代替 AppNavigation。

语义：

```
AppNavigation
→ App 一级页面

AppTabBar
→ 当前页面内部 Tab
```

8. AppSheet

统一处理：

- Bottom Sheet
- Popover
- Context Menu
- Action Sheet

调用：

```
AppSheet.show(
  context,
  type: AppSheetType.adaptive,
  builder: ...
);
```

平台自动选择。

```
iOS
→ Sheet / Popover

Android
→ Material Bottom Sheet

Linux
→ Desktop Menu / Dialog
```

业务代码不得直接调用平台 Sheet API。

9. AppDialog

统一处理确认、警告、错误。

```
final result = await AppDialog.confirm(  
  context,  
  title: '删除照片',  
  message: '确定删除? ',  
);
```

平台：

```
iOS  
→ Apple Alert  
  
Android  
→ Material Dialog  
  
Linux  
→ Desktop Dialog
```

10. Platform Adapter

平台判断必须集中。

```
platform/  
├── ios/  
├── android/  
└── linux/
```

业务代码禁止：

```
Platform.isIOS  
Platform.isAndroid  
Platform.isLinux
```

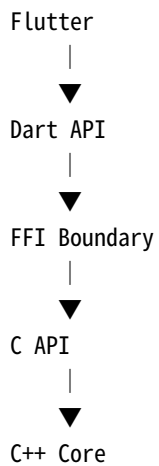
除非属于：

```
core/platform  
design_system/platform  
services/platform
```

平台判断只允许出现在架构边界。

11. Flutter 与 C++ 通信

推荐：

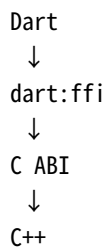


不要让 Dart FFI 直接绑定复杂 C++ ABI。

推荐使用稳定 C ABI：

```
extern "C" {  
  
    void* app_create();  
  
    void app_destroy(void* handle);  
  
    int app_process(  
        void* handle,  
        const uint8_t* data,  
        size_t size  
    );  
  
}
```

Flutter:



12. C++ Core API 设计

C++ Core 必须做到：

Platform Independent

推荐：

```
cpp/
├── core/
│   ├── image/
│   ├── database/
│   ├── filesystem/
│   ├── search/
│   └── media/
├── api/
│   └── c_api.h
└── platform/
    ├── ios/
    ├── android/
    └── linux/
```

核心代码：

core/

不得依赖平台。

平台代码：

platform/

负责：

- 文件系统差异
- 线程差异
- 系统 API
- 平台编码
- 平台路径

13. 内存管理

跨 Flutter / C++ 边界时必须明确：

谁分配
谁使用
谁释放

推荐：

C++ Allocate
↓
C++ Free

不要：

C++ malloc
↓
Dart free

跨语言对象必须明确生命周期。

所有 API 必须考虑：

- NULL
- 生命周期
- ownership
- thread safety
- exception safety
- error code

禁止 C++ 异常直接跨 FFI 边界。

14. 数据模型

推荐：

Flutter
→ UI Model

C++
→ Core Model

FFI
→ DTO / C ABI Struct

不要直接把复杂 C++ Class 暴露给 Dart。

使用：

Plain Struct
Primitive
Handle
Buffer

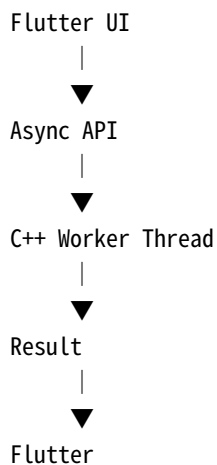
15. 异步设计

耗时任务不得阻塞 Flutter UI Thread。

例如：

- 图片扫描
- 文件扫描
- 数据库查询
- 图片解码
- 缩略图生成
- 搜索索引

必须：

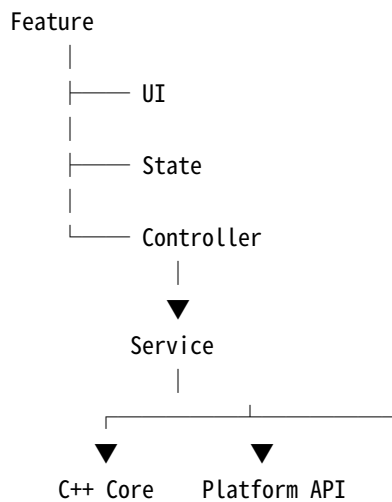


禁止：

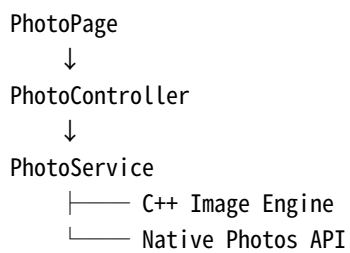
```
build() {  
    heavyCppFunction();  
}
```

16. UI 与功能分离

页面：



例如相册：



UI 不直接调用 C++。

UI 不直接调用 Native API。

17. 原生 UI 使用规则

如果系统已经提供 UI 能力：

优先使用系统 UI。

例如：

iOS Share
→ 系统 Share Sheet

Android Share
→ Android Sharesheet

iOS File Picker
→ 系统 Document Picker

Android File Picker
→ SAF

iOS Photo Picker
→ Photos Framework

Android Photo Picker
→ Android Photo Picker

不要自行重新实现。

18. Native UI 集成规则

Flutter 默认负责：

页面布局
内容展示
业务交互

原生负责：

系统级 UI
系统级交互
平台专属控件
平台专属视觉

需要原生 UI 时：

Flutter
↓
Platform View

↓
Native View

或者：

Flutter
↓
Platform Channel
↓
Native UI Controller

禁止为了追求统一而强行使用 Flutter 模拟系统 UI。

19. iOS 规范

iOS UI 必须优先遵循 Apple Human Interface Guidelines。

优先使用：

SwiftUI
UIKit
Photos
Files
Share
Notifications

Flutter UI：

页面布局
业务 UI
跨平台内容

原生：

系统级 UI
系统级交互
系统服务

Liquid Glass：

App UI
↓
Design System

↓
iOS Adapter
↓
Apple Native Visual Style

不得在业务层直接写：

LiquidGlass(...)

应该：

AppNavigation(...)
AppSheet(...)
AppButton(...)

由 iOS Adapter 处理。

20. Android 规范

Android UI 优先遵循 Material Design。

优先使用：

Material 3
Android Photo Picker
MediaStore
SAF
Android Sharesheet

Flutter：

业务页面
页面布局
跨平台 UI

Android Native：

系统能力
系统 UI
权限
文件

媒体
分享

不要自行实现：

系统文件选择器
系统分享
系统权限页面

21. Linux 规范

Linux 必须考虑桌面环境差异。

优先：

Flutter
↓
Desktop UI

必要时：

GTK
DBus
Desktop API

Linux 不应强行模拟：

iOS UI
Android UI

Linux 默认采用：

Desktop-first

设计。

22. Responsive Design

必须支持：

Phone
Tablet
Desktop

布局根据：

屏幕尺寸
窗口尺寸
输入方式
平台

自适应。

推荐：

Phone
→ Bottom Navigation

Tablet
→ Navigation Rail / Sidebar

Desktop
→ Sidebar

不要简单使用：

```
if (width > 600)
```

到处散落判断。

统一使用：

ResponsiveLayout
Breakpoint
AppScaffold
AppNavigation

23. 路由

页面导航统一管理。

推荐：

AppRouter

业务页面不得直接创建平台原生 Controller。

路由结构：

```
AppRouter
├── Home
├── Albums
├── Photos
├── Folders
└── Settings
```

平台差异由 Router / Navigation Adapter 处理。

24. 状态管理

UI State 与业务 State 分离。

推荐：

```
View
↓
ViewModel / Controller
↓
UseCase
↓
Repository
↓
C++ / Native
```

例如：

```
PhotoPage
↓
PhotoController
↓
PhotoRepository
├── C++ Core
└── Native Photos
```

禁止 Widget 中直接执行复杂业务。

25. 错误处理

所有跨语言 API 必须返回明确错误。

推荐：

```
Success
Error Code
Error Message
```

不要依赖：

```
Exception crossing FFI
```

例如：

```
enum AppError {
  APP_OK = 0,
  APP_INVALID_ARGUMENT = 1,
  APP_IO_ERROR = 2,
  APP_PERMISSION_DENIED = 3,
  APP_NOT_FOUND = 4,
};
```

Flutter 统一转换为：

```
AppException
```

26. 官方 API 优先原则

AI 在实现功能前必须检查：

1. Apple 官方文档
2. Android 官方文档
3. Flutter 官方文档
4. C++ 标准库 / 官方文档
5. 第三方库

对于平台 API：

禁止凭记忆生成 API。

必须确认：

- API 名称
- 最低系统版本
- 生命周期
- 权限
- 异步模型
- 线程要求
- 是否废弃
- 官方推荐替代 API

如果 API 可能发生版本变化，必须查询最新官方文档。

27. AI Coding Agent 行为规范

AI 修改代码前必须：

1. 阅读项目结构。
2. 阅读相关模块。
3. 识别当前架构。
4. 判断功能属于 Flutter / C++ / Native 哪一层。
5. 检查是否已有 Design System 组件。
6. 检查是否已有 Service。
7. 检查是否已有 Platform Adapter。
8. 检查官方 API。
9. 尽量复用现有代码。

AI 禁止：

直接创建重复组件
直接在页面写平台判断
直接在 Widget 中调用 C++
直接在 Widget 中调用 Native API
复制粘贴平台代码
为了快速实现而绕过架构

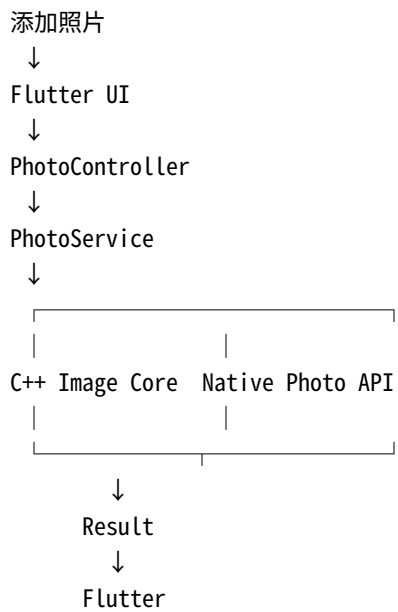
28. 新功能开发流程

任何新功能必须按以下顺序：

需求
↓
确定 UI
↓
确定跨平台逻辑



例如：



29. AI 决策规则

当 AI 面临多个实现方式时：

优先选择

官方 API

其次：

已有项目抽象

其次：

Flutter 官方组件

其次：

平台原生组件

其次：

成熟第三方库

最后：

自定义实现

如果自定义实现可能替代系统能力，必须先确认系统是否已经提供官方能力。

30. UI 组件决策

如果需求是：

普通跨平台 UI

使用：

Flutter

如果需求是：

平台官方 UI

使用：

Platform Adapter

如果需求是：

系统级能力

使用：

Native API

如果需求是：

高性能核心逻辑

使用：

C++

如果需求是：

UI + 高性能计算

使用：

Flutter
↓
C++ FFI

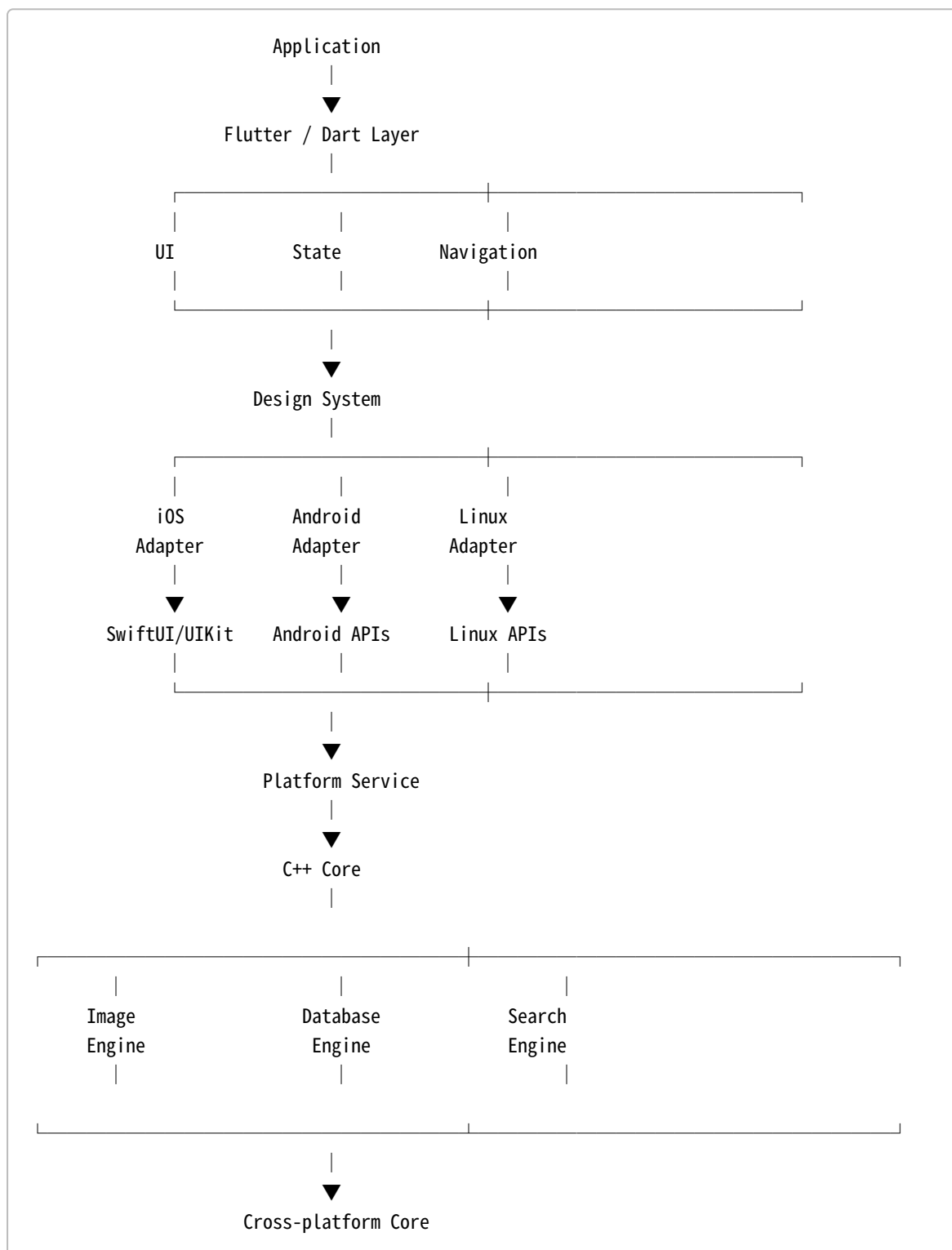
如果需求是：

系统 UI + 系统 API

使用：

Flutter
↓
Platform Channel
↓
Native

31. 最终架构



32. 核心开发哲学

本项目不是：

Flutter App

而是：

Cross Platform Application

Flutter 是：

UI + Application Layer

C++ 是：

Cross Platform Core

Native 是：

Platform Integration

Design System 是：

Platform Experience Abstraction

最终原则：

Flutter

负责「应用是什么」

C++

负责「核心功能怎么运行」

Native

负责「系统如何提供能力」

Design System

负责「用户如何在不同平台体验应用」

必须始终遵循：

统一产品逻辑，不统一平台体验。

共享核心功能，不强行共享系统 UI。

Flutter 负责跨平台，Native 负责平台特色，C++ 负责跨平台核心能力。

优先使用官方系统 API 和官方设计规范，而不是自行模拟系统行为。